

RapidRubric

Group 1

Faculty of Computer Science, Dalhousie University

CSCI4177/CSCI5709 – Advanced Web Services

Dr. Oladapo Oyebode

26th May 2026

1. Title

RapidRubric: AI-Assisted Rubric Grading and Feedback Platform for University Writing Courses.

2. Team Members

The team comprises four members, two undergraduate students, and two graduate students. We have divided key responsibilities by architecture layer so that each person manages one part of the system end-to-end. This avoids knowledge bottlenecks and makes it clear who to contact if some part of the system has issues. All members contribute to shared tasks such as code review, integration testing, and final demo. The primary responsibilities of each member are listed below:

Name	Banner ID	Key Responsibilities
Satwik Gorla	B00941509	<u>Frontend Lead:</u> React + Vite project setup, Tailwind CSS design, React Router navigation, role-based UI views, responsiveness and accessibility compliance, version history viewer, and Netlify Deployment.
Devanshu Mundhiyara	B01053721	<u>Backend Lead:</u> Node.js + Express API design, REST endpoints, PDF-parsing pipeline, submission and rubric management logic, and Railway Deployment.
Sparsh Ladani	B00927926	<u>Database & Auth Lead:</u> PostgreSQL schema design on Supabase, Supabase Auth integration, role-based access control, file compression and storage on Supabase, analytics queries and audit logging .
Dhrumil Hiren Gajjar	B01071041	<u>AI Integration & DevOps Lead:</u> Anthropic Claude API integration, prompt engineering, version-difference analysis, email notification system using Resend, CI/CD Pipeline setup.

3. Problem Statement and Motivation

a) What problem are you solving?

Grading writing assignments in large university courses is slow, repetitive work. A Teaching Assistant (TA) would generally be assigned thirty to forty submissions that they would need to comprehend, apply the rubric to, grade and generate meaningful feedback. A lot of the grading process, such as checking citations, structure and grammar, would be the same across all submissions with slight variations in feedback for each.

On the students' side, after they submit their assignments, they are met with a long wait for their grade and feedback to be returned. By the time they receive the feedback, they may be too far into the next assignment for it to be actionable anymore. Students also face a consistency issue; different TAs may interpret and apply the rubric differently; the same TA may also change their interpretation and application of the rubric towards the end of their assigned submissions compared to the beginning.

Instructors face the problem of not being able to tell how the class is doing overall, and which parts of assignments are giving students the most problems. They also do not know how well the TAs are faring with their assigned workload and provided rubric. Right now, instructors must either ask the TAs for this information or go through submissions themselves.

RapidRubric attempts to address the above issues faced by students, TA's and instructors by looping AI into the grading process. AI scores the submitted assignments and generates feedback according to rubric criteria provided. These grades and feedback are passed along to the TA who reviews them and edits them as necessary to have the final say on grades and feedback before releasing them to the students. The goal of RapidRubric is not to replace TA's and take over the grading and feedback work, but to help speed up the process and allow TA's to better utilize their time to help support the students.

b) Why does it matter?

A major pain-point faced by students is turnaround time from when an assignment is submitted to when feedback and grades are returned. If feedback is provided too late, it may no longer be actionable. Faster grading means students have more time to act on the feedback and learn from it to do better in the writing course. Consistency is also another pain-point that we address, if the AI makes its first pass over the submissions using the rubric, all the grades and feedback are results of the same uniform evaluation process. The TAs can then make necessary changes as required, but all of them start from the same baseline, reducing inconsistencies.

For TA's a significant part of repetitive work is taken off their shoulders, while they still keep full authority over final grades and feedback that is sent out, they will no longer need to type out nearly identical feedback over multiple students anymore, allowing them to focus on more specialized feedback for each student.

Instructors can find answers to certain questions such as class performance, pain points faced by students in assignments, and how far along grading has come without having to ask TAs. This helps them decide what concepts to focus on in upcoming lectures and determine if TAs are struggling with their currently assigned workload.

c) Who is affected?

RapidRubric is built around the three common roles found within writing courses:

Students: They can submit written assignments to the platform, receive grades and feedback once the TA's release them, view version history, and resubmit revised work if the assignment is set to allow it.

Teaching Assistants: They receive the submissions along with the grades and feedback generated by AI's first pass. They can then make any necessary edits to grades and feedback to have the final say before releasing them to the students.

Instructors: They have complete ownership over course setup and assignment configuration. They make the rubrics, assign TAs to submissions, have access to a comprehensive class-wide analytics dashboard to monitor performance, and reserve the ability to override any grade and feedback.

4. Objectives

RapidRubric has five primary objectives:

1. Cut down on the repetitive parts of the grading process: AI does a first pass over the submissions according to the rubric to produce initial scores, flag issues and draft feedback. This allows TAs to focus on making calls and judgements instead of applying and writing similar feedback to many submissions. This also has the effect of reducing bias that comes with seeing the work of prior students before the current one throughout the grading process.
2. Set a formalized feedback pipeline: Structure the submission process around a clear, unambiguous pipeline: students submit work -> AI pre-grades -> TA reviews, makes edits and releases them -> students receive grades and feedback. No AI generated output reaches the students before the TA reviews and makes necessary edits. This significantly reduces turnaround time from when a student submits work to when feedback and grades are released.
3. Visualize and Evaluate improvement: On resubmission enabled assignments, generate a version difference report that shows how the student has improved, regressed or kept their standing per rubric criteria compared to their last submission. Right now, students may not be able to tell if their revisions properly addressed the initial feedback they got; the resubmission system makes sure to highlight the impact these revisions had.
4. Instructors are given a bird's eye view of the class: Analytics are generated through aggregating student scores across rubrics which help identify what concepts the class is finding difficulty with. The main goal is to let instructors effectively make changes to their lecture content and assignments through exact data rather than making educated guesses.
5. Incorporate access control and logging: Students are only allowed to access the submission portal and rubric, TA's only see the submissions and the AI generated grades and feedback assigned to their queue, and instructors see and control everything in the course. Any feedback or grade changes leave a trail in the form of audit logs, providing accountability.

5. Scope and Deliverables

a) Core features

Accounts: Users can register and sign in through Supabase Auth. Students, TAs and instructors each have an account tied to a role which determines their access level (Role Based Access Control). Instructors can assign and reassign TA roles. The role determines what is visible to the user across the whole system.

Draft Submission and File Upload: Students see a page listing open assignments for their writing course. They can select one, upload a PDF, add any optional comments, and submit. Deadline cutoffs and file size limits are enforced on both the client and server, so the frontend cannot be manipulated to bypass these restrictions. We aim to make the frontend feel familiar to Brightspace as its workflow is something many students are already familiar with.

Rubric Builder for Instructors: Instructors are provided with tools to create rubrics in the form of an ordered list of criteria, each of which has a name, a description and point weightage. The rubric is locked to edits once the grading process starts so that students are all graded against the same rubric without it

being modified midway through the grading process. Only the instructor has the authority to make and edit the rubric.

TA Review and Approval Queue: After the first pass is completed by AI, the resulting grades and feedback along with the submissions are placed in the assigned TA's review queue. The TA gets to review the AI-generated grades and feedback side by side to the submitted PDF and given rubric. They have the authority to change grades or feedback. The TA then must intentionally release feedback and grades to the students by deliberately clicking a button. This ensures that AI-generated scores and feedback cannot reach the student before the TA reviews, edits, and signs off on it.

Feedback and Grade Viewer: Once a TA releases feedback, students can open a viewer that shows their submission along with their final score and feedback broken down according to the rubric. On resubmissions, the version difference report is also displayed which shows what has changed between their current and previous draft with relevant feedback.

Version History Viewer: For assignments with resubmitting enabled, students and TAs can look through every version of submissions chronologically along with feedback and grades attached to them. The AI also accesses the backend of this system to make comparisons between the current draft and previous drafts submitted by the student.

Email Notifications: The system sends emails at specific points of the process: when an instructor assigns the TA to an assignment submission queue (TA gets notified), when a student submits their work (assigned TA gets notified), when TA releases feedback (notification gets sent to student), when an instructor reassigns work from one TA to another (both TAs get notified), and when all submissions have been evaluated and all feedback and grades released (instructor gets notified). Notifications are also buffered for a short period of time so that a large amount of email notifications is sent to an email address for many notifications in rapid succession; they are all sent as one email instead.

b) AI-Enabled Features

Rubric-based AI grading and Feedback: When a student submits a PDF, the backend extracts the text from it using PDF-parse, packages it with the rubric set and sends it to the Claude API. Claude returns grades and short feedback per criteria of the rubric, a list of flagged potential issues, and a short summary of the feedback. This output is attached to the submission and placed in the assigned TA's queue; there is nothing being sent directly to the student.

Version History Analysis: for assignments with resubmission enabled, when a student submits a revised draft, the backend also attaches the previous version of the draft submitted with its final score and feedback and sends the package to Claude to compare against the revised draft. Claude then, in addition to grading it like the Rubric-based AI grading and Feedback feature, it also labels if the new revision of the draft has improved or regressed according to the criterion present in the rubric and the feedback for the previous draft. This makes sure that the feedback given for the previous draft is considered when giving grades and feedback for the current draft.

Class-Wide Weakness Detection: On the instructor dashboard, the application provides information regarding the rubric criteria with the lowest mean score across the class alongside an AI-generated summary of the reasons given by TAs for the low score across the submissions in the feedback. The goal

is to provide the instructor with actionable information representing the whole picture, so the instructor has a clear picture of exactly what is going on.

c) Out of Scope

To keep the project completable within the eight-week timeframe, the following commonly found features among education platforms are not being considered: plagiarism detection, direct integration with existing platforms such as Brightspace, real time collaborative editing on a single submissions feedback and grades, and submission and grading of non-writing assignments such as images, video or code based deliverables.

6. Technical Approach

The application is built around three independently deployed layers, the frontend client, the backend API, and hosted database and file storage. The communication between these three parts is through well-defined HTTP interfaces.

a) Frontend

We will be using React with Vite as the build tool, Tailwind CSS for styling and React Router for client-sided navigation. Most of the tech stack is familiar with the team (except for Tailwind CSS which was chosen due to its prevalence in industry and recommendation by some of the members). The frontend will be deployed to Netlify as it features automatic deployment using a Git repository and is free to use for our needs.

b) Backend

The API server will be built with Node.js and Express. It has been chosen as such to pair well with the React frontend through the use of REST API endpoints. It is also lightweight, well documented, and familiar with the team. We will be using the PDF-parse npm library to extract text from uploaded PDFs for processing before sending them to Claude. All API calls made to the endpoint will be verified through Supabase Auth to prevent unauthorized access. The system will be deployed to Railway which also auto deploys from Git.

c) Database and File Storage

All relational data related to users, courses, assignments, submissions, AI Feedback, TA reviews, version history, aggregated data for analytics and audit logging will be contained in a PostgreSQL database hosted on Supabase which features a free plan with a small amount of free storage which will be enough for the scope of this project. Uploaded PDFs will also be stored on Supabase in a compressed form with keys pointing to them stored in the database for easy linking.

d) External Services

Anthropic Claude API will be used to handle all the work related to the AI enabled feature (rubric based grading, version history analysis, and class-wide weakness detection reports). Claude supports sending in a long prompt that comprises the student submission, rubric and previous draft (if the assignment allows for resubmission). We will be using anthropic free credits provided and monitoring credit cost per call as

part of DevOps. We will also be mocking AI responses during development and testing as to not use up credits when other related features are being worked on to avoid wasting credits when not needed.

Another external service being used is Resend, which will be used to send email notifications triggered on certain key events (release of feedback and grades will send an email notification to the students for example).

e) Separate Deployment and Communication Pipeline

The frontend is built with Vite and deployed onto Netlify and contacts the backend through API. The backend is built using Node.js + Express and deployed to Railway. It handles authentication of requests sent to its REST API endpoint and holds the Claude API key; it is the only system that can communicate with Claude. The database is hosted on Supabase and is accessible only from the backend. All the sensitive keys and logic are hosted on the backend to maintain security.

7. System Architecture

The diagram below shows the three deployed layers, the three user roles, the core features, the AI features, and the external services, with all the connections between them. Solid blue lines are REST/JSON calls between the client and the server; purple lines are database and file-storage I/O from the backend; red lines are HTTPS calls from the application to external services. Plain black arrows are users interacting with the UI.

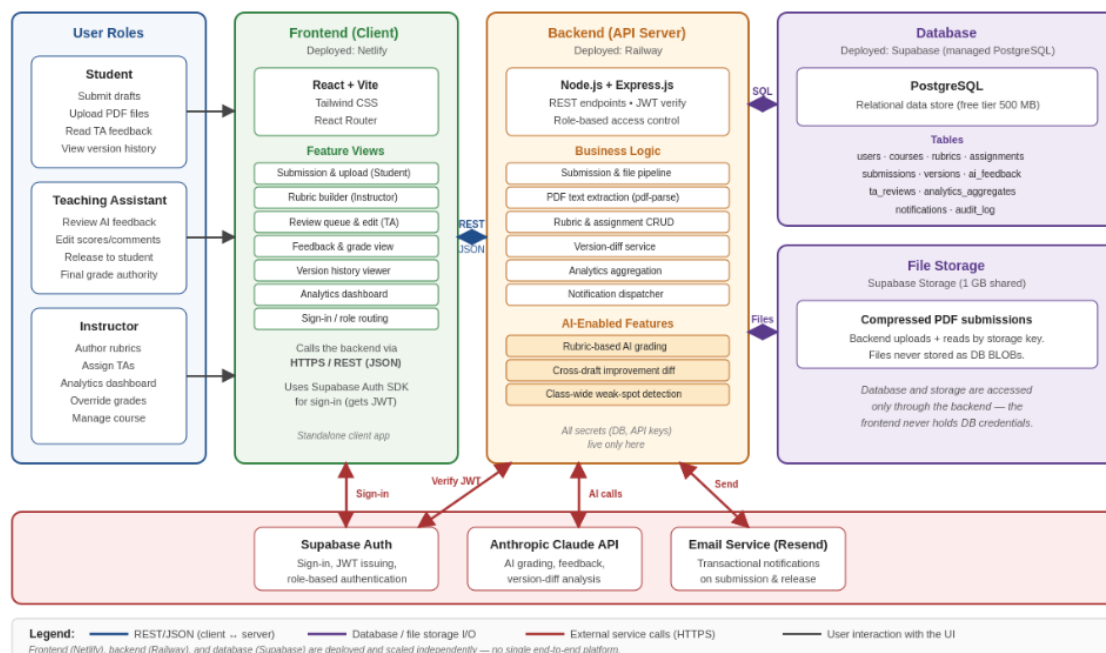


Figure 1. RapidRubric System Architecture

Data flow Starting from Student Submitting Draft:

1. The student signs in through Supabase Auth and receives a JWT, which the frontend stores.
2. The student uploads a PDF through the React submission view. The frontend sends the file and the JWT to the backends' POST /submissions endpoint.
3. The backend verifies the JWT, confirms the user's role (Student) and that the assignment is still open, stores the compressed file in Supabase Storage, and writes a row to the submissions table in PostgreSQL.
4. The backend extracts text with pdf-parse, fetches the rubric and (if applicable) the previous version, and calls the Claude API with a prompt assembled from all three.
5. Claude returns per-criterion scores, flagged issues, drafted feedback, and the version-diff (when applicable). The backend writes the result to ai_feedback with status = pending_ta_review.
6. The backend asks Resend to email the assigned TA that a new submission is in their queue.
7. The TA opens the queue, edits the AI output as needed, and presses Release. The backend updates the row to status = released, updates the analytics aggregates, and asks Resend to email the student.
8. The student opens the feedback view, which fetches the released feedback from the backend and renders it alongside the version-diff.

8. Timelines and Tasks

The work is structured across eight weeks. We deliberately wired the schedule so that an end-to-end deployed version of the app exists from very early on (Week 2), even if it doesn't do anything interesting yet. This is to avoid failure mode where everything works in isolation, but nothing works together until the last week. Mocked AI output is used for Weeks 3 and 4, so the full workflow can be tested before the real Claude integration goes in.

a) Weekly Milestones

Week	Milestone / Theme	What gets done
Week 1	Foundations	Wireframes finalized. Repos created. Vite + React + Tailwind scaffold. Express server scaffolds up. Supabase project created. First draft of the API contract. CI pipelines are configured.
Week 2	Auth & deploy skeleton	Supabase Auth wired in. JWT verification middleware on the backend. Frontend deployed to Netlify, backend to Railway. End-to-end sign-in works. Milestone: deployed hello-world app with working auth.

Week 3	Submission flow & rubric builder	Student submission UI built. File upload endpoint with Supabase Storage. PDF parsing with pdf-parse. Rubric builder UI for instructors. Rubric CRUD endpoints with the locking rule.
Week 4	TA queue with mock AI	TA review queue and side-by-side review UI. Release endpoint. Submission state machine. Mock AI output flows through the whole pipeline. Milestone: Full workflow works end-to-end with mock AI.
Week 5	Real AI integration	Claude API integrated. Grading prompt iterated against sample submissions. Response parsing and validation. Citation / issue flagging logic. Mocks replaced with real calls.
Week 6	Version-diff & analytics	Version-diff prompt and service. Analytics aggregation. Instructor dashboard UI with charts. Class-wide weak-spot summarization. Milestone: all AI features functional.
Week 7	Notifications, polish, testing	Resend integrated, and templates written. Notifications trigger on the key events. UI polish and accessibility pass. Integration tests for submission → review → release. Synthetic load test with 100+ submissions.
Week 8	Hardening, docs, demo	Bug fixes from testing. Role Based Access Control (RBAC) tightened. User documentation. Demo video recorded. Final deployment freeze. Milestone: project submission.

b) Per-Person Task Breakdown

The table above shows team-level milestones. The breakdown below shows what each of us is on the hook for each week. This is also how we track who is the point of contact if a piece of work is blocked or running late.

Week	Satwik (Frontend)	Devanshu (Backend)	Sparsh (DB/Auth)	Dhrumil (AI/DevOps)
1	Vite + React + Tailwind scaffold; routing skeleton;	Express scaffold; module structure;	Supabase project; initial schema; first migration; ERD draft.	GitHub repos; CI pipeline for both

	placeholder pages per role.	first draft of the API contract document.		layers; first draft of the grading prompt.
2	Sign-in / sign-up pages; role-aware route guards; Netlify deploy.	JWT verification middleware; protected user endpoint; Railway deploy.	Users / roles tables; Supabase Auth wiring; seed data for dev.	Env-var management across all three environments; deploy smoke tests.
3	Student submission UI; instructor rubric builder UI.	POST /submissions with file upload; pdf-parse pipeline; rubric CRUD endpoints.	Rubric, submission, and version tables; Supabase Storage policies.	Iterate the grading prompt against sample submissions.
4	TA review queue + side-by-side review/edit UI.	Review and release endpoints; submission state machine.	ai_feedback and ta_reviews tables; release-triggered analytics update.	Wire mock AI output through the pipeline so the full flow is testable.
5	Polish review UI after first integration tests; fix uncovered bugs.	AI orchestration service in backend; replace mocks with real calls.	Indexes for queue and dashboard queries; tune slow paths.	Claude API integration; response validation; citation / issue flagging.
6	Version history viewer; analytics dashboard UI with charts.	Version-diff service; analytics aggregation routines.	analytics_aggregates table; update triggers.	Version-diff prompt; weak-spot summarization prompt.
7	Accessibility pass; UI polish; mobile responsiveness fixes.	Notification triggers; integration tests for the full pipeline.	audit_log triggers; synthetic load test with 100+ submissions.	Resend integration; email templates; AI regression test harness.
8	Final UI bug fixes; record student / TA / instructor walkthroughs.	Final backend bug fixes; API documentation.	Backup / restore drill; final schema documentation.	Demo video; README; final deployment freeze.

c) Risk Mitigation Built into the Schedule

- Reaching an end-to-end deploy by Week 2 means integration issues surface early, when there is still time to fix them.
- Mock AI output covers Weeks 3 and 4, so the non-AI workflow is fully working before we depend on the Claude API at all.
- Week 7 is mostly polish and testing, which gives us a buffer if earlier weeks slip.
- The two AI weeks (5 and 6) are decoupled. If grading takes longer than expected, the version-diff feature can be scoped down without affecting the core submit-and-review flow.